

# 모두를 위한 딥러닝 (입문편)

허 찬

# 강의 계획표 - 모두를 위한 딥러닝 (입문편)

| 학습주제(강좌명)              | 차시(모듈) | 차시별 주제                  | 학습내용                                                                     |
|------------------------|--------|-------------------------|--------------------------------------------------------------------------|
| 모두를 위한<br>딥러닝<br>(입문편) | 1      | Dive into deep learning | 머신러닝, 딥러닝, 인공신경망의 개념 이해<br>딥러닝의 분류 및 응용 분야 이해<br>딥러닝 모델의 장/단점 이해         |
|                        | 2      | 딥러닝 코딩 환경 준비            | 머신러닝/딥러닝 워크플로우의 이해<br>파이썬 기반 머신러닝, 딥러닝 프레임워크의 이해<br>패키지 설치 및 기초 벡터 연산 설명 |
|                        | 3      | 딥러닝 모델 학습의 4steps       | 딥러닝 학습 과정의 단계별 이해                                                        |
|                        | 4      | 데이터 시각화                 | 데이터 시각화<br>Pandas, Matplotlib, Seaborn 패키지의 사용                           |
|                        | 5      | 딥러닝을 이용한 분류 모델 실습       | 딥러닝 코드 실습을 통한 분류 문제 해결                                                   |
|                        | 6      | 딥러닝을 이용한 회귀 모델 실습       | 딥러닝 코드 실습을 통한 회귀 문제 해결                                                   |
|                        | 7      | 딥러닝 모델의 성능 향상 기법        | 딥러닝 기본 모델의 성능 향상 기법                                                      |

# 머신 러닝/딥러닝 workflow

데이터 -> 데이터 전처리 -> **EDA (데이터 탐색)** -> 모델 구축 -> 하이퍼파라미터 튜닝

1. 데이터 : 스프레드시트, 이미지, 텍스트, 비디오, 오디오...
2. 데이터 전처리 : 데이터 결측치 해결, normalization, 데이터 증강, train/test split
3. **EDA : 데이터 탐색 작업 (시각화 , 분석...)**
4. 모델 구축 : Decision tree, KNN, Logistic regression, Neural network...
5. 하이퍼파라미터 튜닝 : 성능 향상을 위한 파라미터 조절

## 4차시

---

|     |         |                                                |
|-----|---------|------------------------------------------------|
| 1.4 | 데이터 시각화 | 데이터 시각화<br>Pandas, Matplotlib, Seaborn 패키지의 사용 |
|-----|---------|------------------------------------------------|

### Contents

- 왜 데이터 시각화가 필요한가?
- 데이터의 시각화
- 데이터프레임의 이해 및 사용
- 시각화된 데이터의 분류

## 1.4 데이터 시각화

### 데이터 시각화란?

- 데이터를 시각적인 그래픽으로 변환하여 패턴, 트렌드, 관계, 이상치 등을 이해하고 전달하는 과정. 이는 복잡한 데이터를 직관적으로 이해할 수 있도록 도와주는 강력한 도구 중 하나.
- 엑셀, R, 파이썬 등에서 많이 사용되는 도구 중 하나.



## 1.4 왜 데이터 시각화가 필요한가?

### 1. 복잡한 패턴 및 관계의 시각적 이해

- 데이터는 종종 복잡한 패턴과 다양한 변수 간의 상관 관계를 포함합니다. 시각화는 이러한 복잡성을 간단한 차트, 그래프 또는 플롯으로 변환하여 직관적으로 이해할 수 있도록 도와줍니다. 예를 들어, 선 그래프나 산점도를 통해 시간에 따른 추세나 변수 간의 상관 관계를 파악할 수 있습니다.

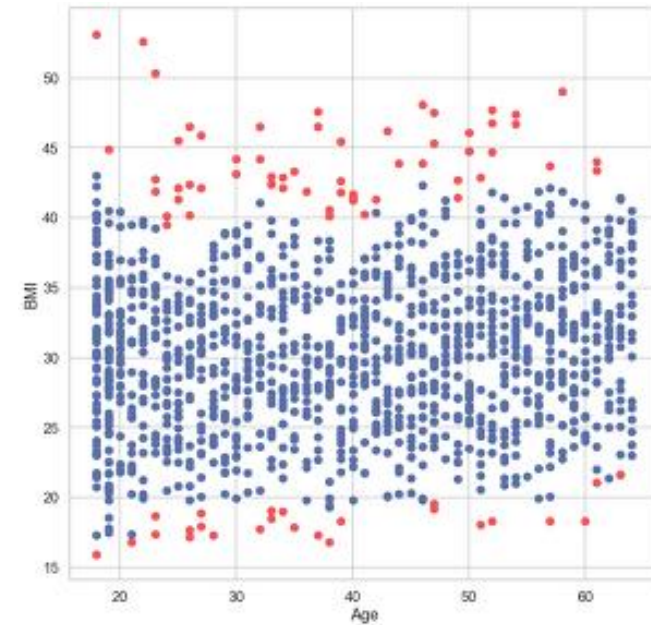
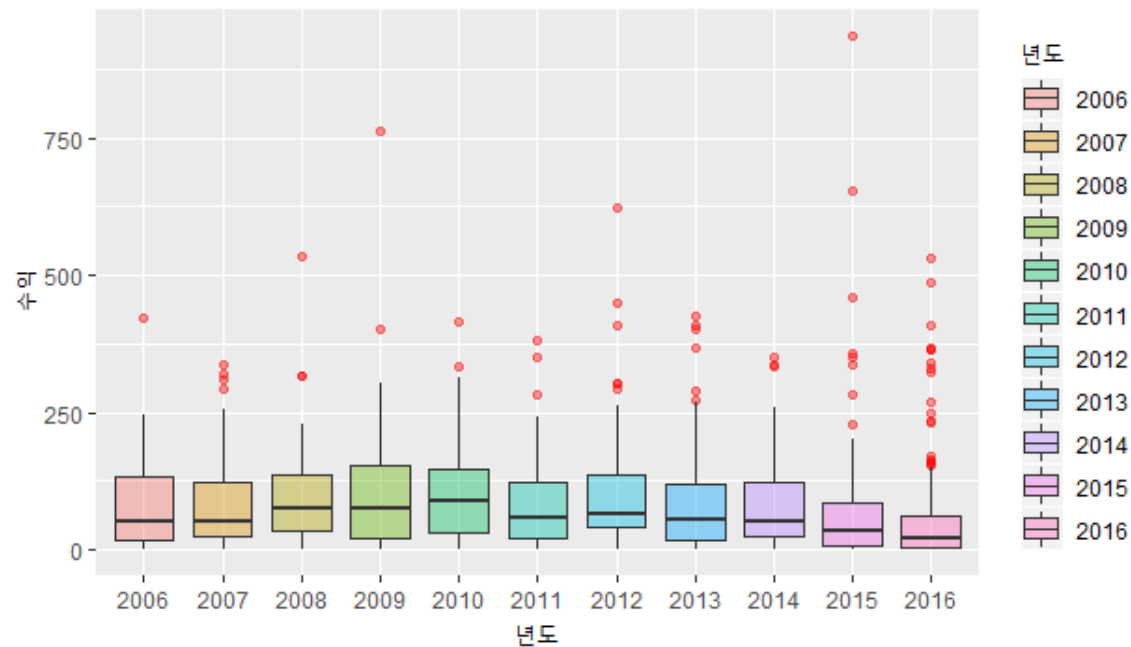
채널 중점추진사항 진도현황 [요약]



# 1.4 왜 데이터 시각화가 필요한가?

## 2. 패턴 및 이상치 탐지

- 데이터 시각화는 데이터에서 패턴을 식별하고 이상치를 감지하는 데 도움이 됩니다. 시각적으로 이상치를 확인하면 데이터의 문제점을 더 빨리 파악하고 수정할 수 있습니다. 상자그림(boxplot)이나 히스토그램을 통해 데이터 분포의 특이한 부분을 시각적으로 확인할 수 있습니다.

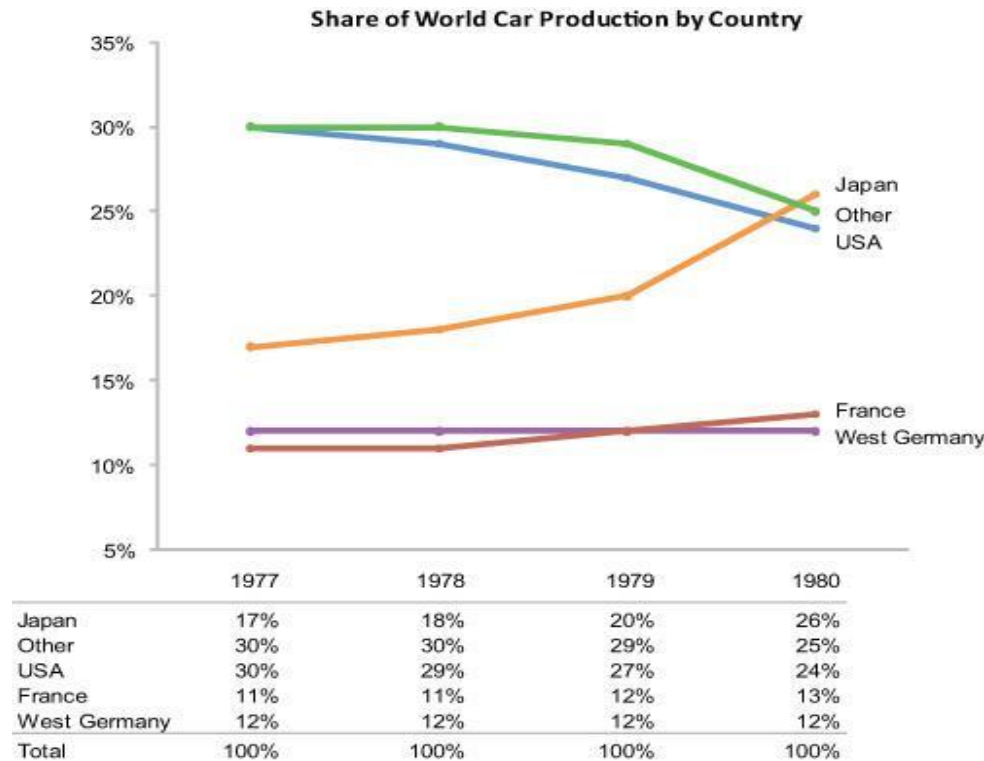


극단적인 데이터를 눈으로 빠르게 확인할 수 있다!

## 1.4 왜 데이터 시각화가 필요한가?

### 3. 의사소통과 결정 지원

- 시각화는 데이터를 다른 이해관계자들과 공유하고 이야기를 풀어내는 데 도움이 됩니다. 간결하고 명확한 시각화는 의사소통을 향상시키며, 비즈니스 의사 결정에 필수적인 정보를 제공합니다. 대시보드, 차트, 그래프 등을 통해 핵심 인사이트를 빠르게 전달할 수 있습니다.



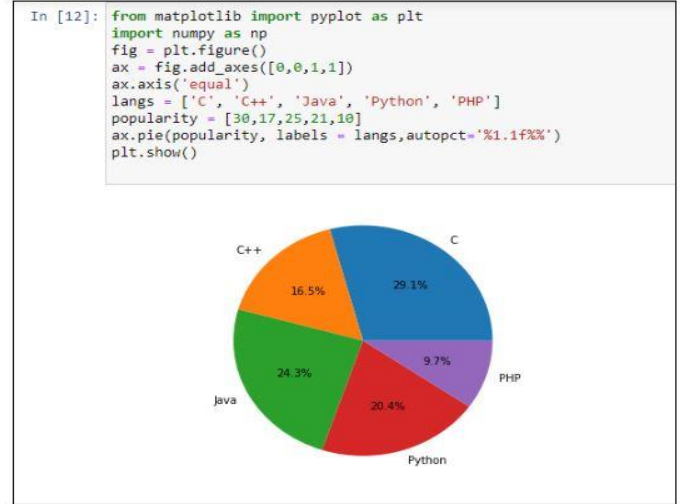
수치데이터 vs 시각화 자료



## 1.4 왜 데이터 시각화가 필요한가?

### 4. 데이터 탐색과 탐구

- 데이터 시각화는 데이터를 탐색하고 새로운 인사이트를 발견하는 데 도움이 됩니다. 시각적 도구를 사용하면 데이터의 구조, 분포 및 트렌드를 빠르게 파악할 수 있으며, 이는 데이터 과학자나 분석가가 데이터를 더 깊이 탐색하는 기초를 제공합니다.



## 1.4 데이터 시각화 Tool

### pandas 및 matplotlib, seaborn 개요

- 데이터 분석에서 널리 활용하는 파이썬 라이브러리.
- 엑셀 등에서 숫자로 기입된 수치형 자료를 table 형태로 표현하는 방식이 많음
- **Pandas** 라는 패키지에서 Dataframe이라는 표현형식을 이용하여 table 형태 데이터를 시각화 및 머신러닝을 위한 데이터셋으로 변환.
- **Matplotlib, seaborn** 패키지를 통해 자료들을 시각화.

```
In [1]: import pandas as pd  
file_name = "https://people.sc.fsu.edu/~jburkardt/data/csv/homes.csv"  
df = pd.read_csv(file_name)
```

```
In [2]: df.head()
```

```
Out[2]:
```

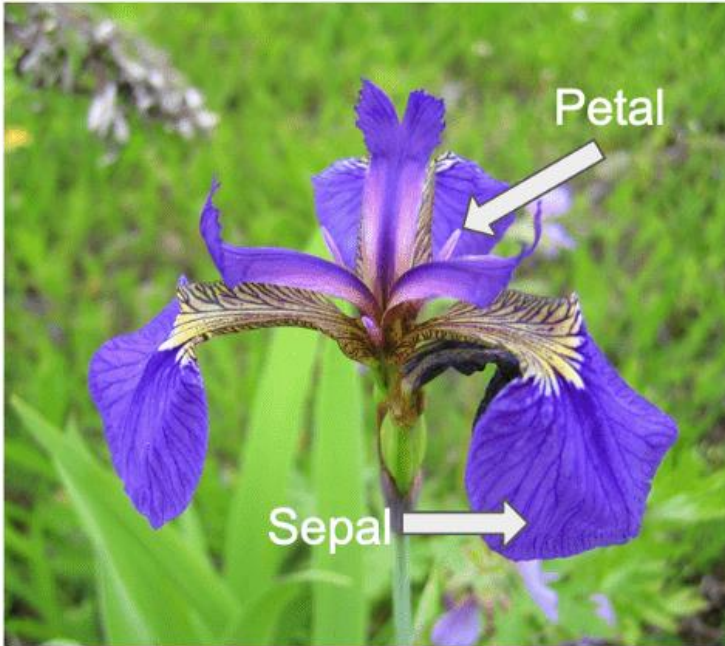
|   | Sell | "List" | "Living" | "Rooms" | "Beds" | "Baths" | "Age" | "Acres" | "Taxes" |
|---|------|--------|----------|---------|--------|---------|-------|---------|---------|
| 0 | 142  | 160    | 28       | 10      | 5      | 3       | 60    | 0.28    | 3167    |
| 1 | 175  | 180    | 18       | 8       | 4      | 1       | 12    | 0.43    | 4033    |
| 2 | 129  | 132    | 13       | 6       | 3      | 1       | 41    | 0.33    | 1471    |
| 3 | 138  | 140    | 17       | 7       | 3      | 1       | 22    | 0.46    | 3204    |
| 4 | 232  | 240    | 25       | 8       | 4      | 3       | 5     | 2.05    | 3613    |

## 1.4 데이터 시각화

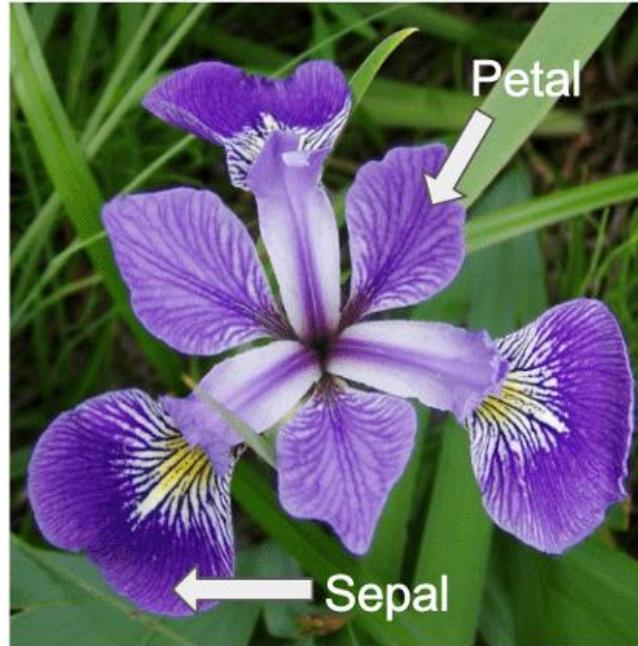
### 데이터셋 개요

### 데이터 시각화 in Iris(붓꽃) data

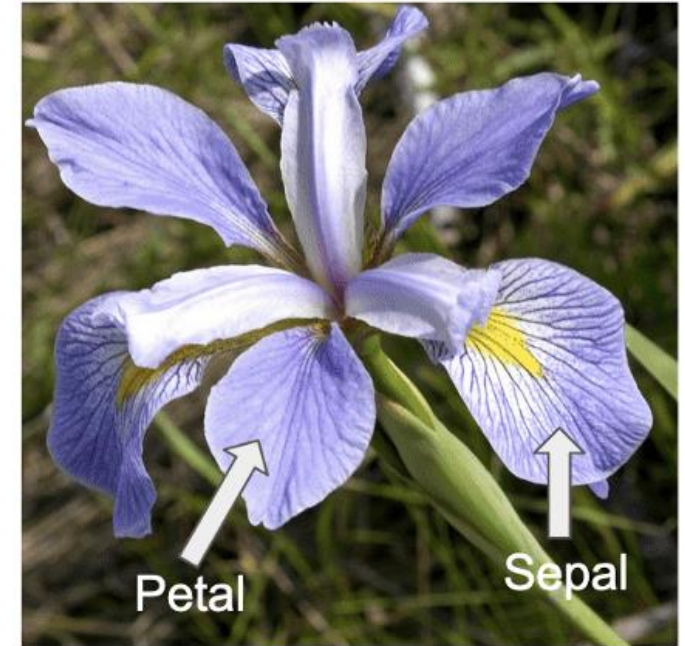
*Iris setosa*



*Iris versicolor*



*Iris virginica*



- Iris(붓꽃) 데이터셋의 시각화 및 분류 실습

## 1.4 데이터 시각화 in Iris(붓꽃) data

### 1. Module import

```
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
```

## 1.4 데이터 시각화 in Iris(붓꽃) data

### 2. Data load & Check

```
# UCI 데이터베이스에서 붓꽃 데이터셋을 가져온다
df = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data",
                 names = ['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'class'])
#sepal_length : 꽃받침 길이
#sepal_width : 꽃받침 폭
#petal_length : 꽃잎 길이
#petal_width : 꽃잎 폭
#class : 분류하고자 하는 class (3개, vesicolor/virginica/setosa)

# 데이터 유형을 살펴본다
print(df.info())
print('')
```

#### **pd.read\_csv 함수:**

- 수치형 데이터 파일(csv)를 dataframe이라는 pandas의 자료형태로 변경
- 데이터프레임 자료구조형은 굉장히 많이 사용되는 구조!

## 1.4 데이터 시각화 in Iris(붓꽃) data

### 2. Data load & Check

```
#데이터프레임의 정보를 요약한다
df.describe()
```

```
# 1~10번 데이터를 확인한다
df.head(10)
```

```
# sepal_length가 5.0보다 큰 로우만 선택한다
df2 = df.loc[df['sepal_length'] > 5.0]
print(df2)
```

### 코드 설명

**df.describe() :**

- Dataframe의 정보를 확인하는 함수.

**df.head(num):**

- Dataframe 안의 몇 개 샘플을 볼것인가 결정하는 함수.

**df2 = df.loc[조건] :**

조건을 만족시키는 행만 가져와서 새로운 dataframe을 만드는 함수

## 1.4 데이터 시각화 in Iris(붓꽃) data

```
'''
```

```
판다스를 활용한 데이터 시각화
```

```
'''
```

```
# 클래스별로 마커 모양을 지정한다
```

```
marker_shapes = ['.', '^', '*']
```

```
# 점차트를 그린다
```

```
for i, species in enumerate(df['class'].unique()):
```

```
    if i == 0:
```

```
        ax = df[df['class'] == species].plot.scatter(x='sepal_length', y='sepal_width',
                                                    marker=marker_shapes[i], s=100, title="Sepal Width vs Length by Species",
                                                    label=species, figsize=(10,7))
```

```
    else:
```

```
        df[df['class'] == species].plot.scatter(x='sepal_length', y='sepal_width', marker=marker_shapes[i],
                                                s=100, title="Sepal Width vs Length by Species", label=species, ax=ax)
```

```
plt.show()
```

```
# 히스토그램을 그린다
```

```
df['petal_length'].plot.hist(title='Histogram of Petal Length')
```

```
plt.show()
```

```
# 상자차트를 그린다
```

```
df.plot.box(title='Boxplot of Sepal Length & Width, and Petal Length & Width')
```

```
plt.show()
```

## 코드 설명

**for i,species in enumerate(df['class'].unique()):**

- enumerate라는 python 함수는 인덱스와 함께 반복 요소를 return

**df.plot.scatter/df.plot.hist/df.plot.box:**

- 산점도, histogram, boxplot을 그려주는 함수

**plt.show():**

- 화면에 실제로 작성한 figure를 출력하는 함수

## 1.4 DNN을 이용한 Iris data 분류

### 3. 학습 데이터셋 만들기

```
from sklearn.preprocessing import LabelEncoder

# 학습 데이터와 label을 분류
X = df.iloc[:,0:4].values
Y = df.iloc[:,4].values

encoder = LabelEncoder()
y1 = encoder.fit_transform(y)
print(y1)
Y = pd.get_dummies(y1).values #one-hot encoding
print(Y)
```

### 코드 설명

**LabelEncoder() / encoder.fit\_transform:**

- 자동으로 label을 할당해주는 함수 (0~2)

**pd.get\_dummies 함수:**

- One-hot encoding으로 만들어주는 함수



## 1.4 DNN을 이용한 Iris data 분류

### 4. 학습&테스트 데이터셋 나누기

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, Y,
                                                    test_size=0.2,
                                                    random_state=1)

X_train.shape, X_test.shape, y_train.shape, y_test.shape
```

#### 코드 설명

##### **train\_test\_split 함수:**

- 전체 dataset을 train용 / test용으로 나누는 함수.
- 비율을 test\_size로 조절하는데 보통 train:test를 80:20으로 정함

## 1.4 DNN을 이용한 Iris data 분류

### 5. Model build

```
from keras.models import Sequential
from keras.layers import Dense
from keras.optimizers import Adam

model = Sequential()

model.add(Dense(64, input_shape=(4,), activation='relu'))
model.add(Dense(64, activation='relu'))
model.add(Dense(3, activation='softmax'))
model.summary()
```

## 1.4 DNN을 이용한 Iris data 분류

### 6. Compile & training

```
model.compile(loss='categorical_crossentropy',  
              optimizer='Adam',  
              metrics=['accuracy'])  
model.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=50, verbose=1)
```

### 코드 설명

#### **validation\_data 옵션:**

- 학습 중에 test dataset에서의 정확도를 볼 수 있는 옵션

## 1.4 DNN을 이용한 Iris data 분류

### 7. Test 결과 확인

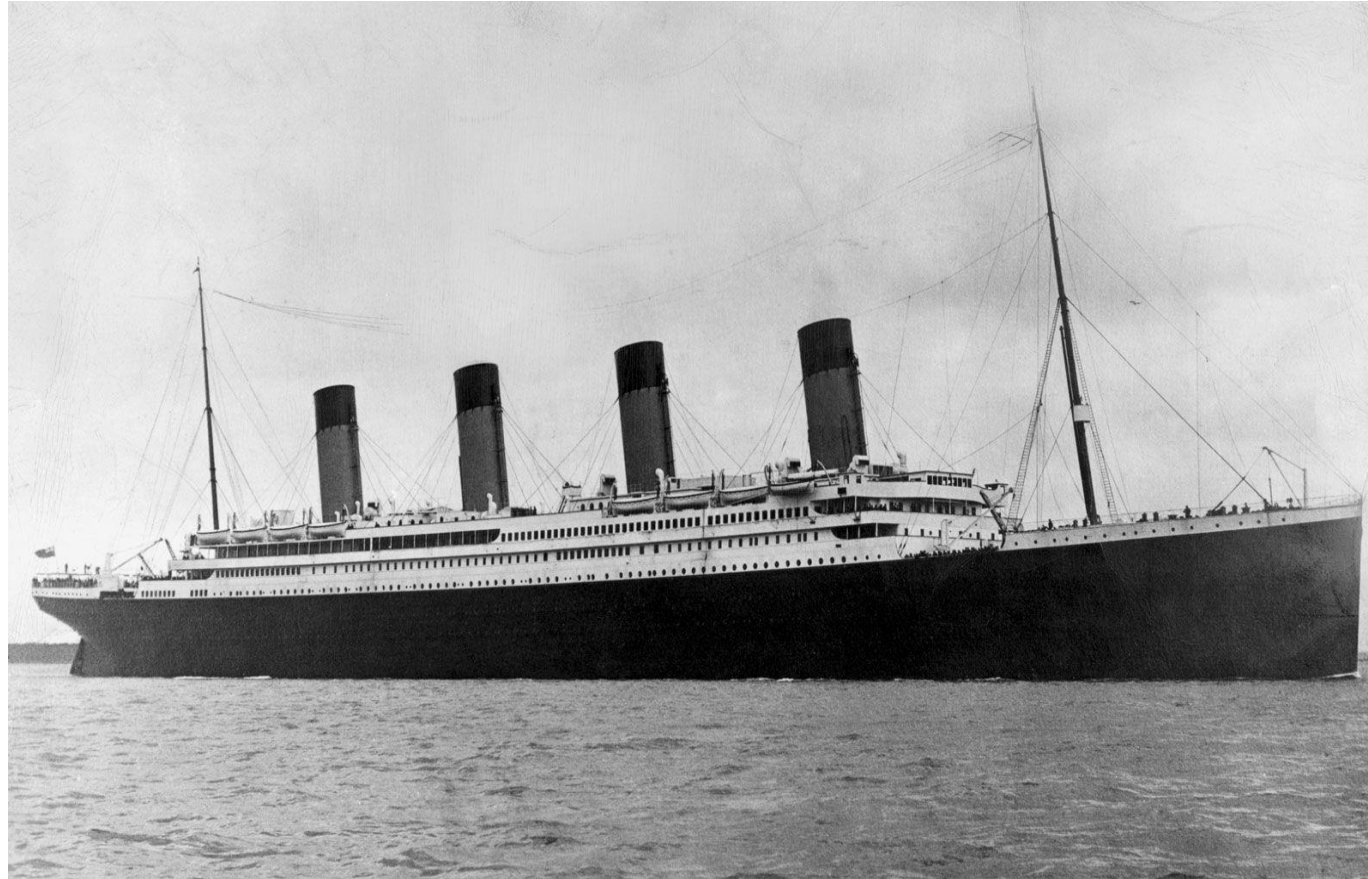
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,8))
plt.plot(hist.history['loss'])
plt.plot(hist.history['val_loss'])
plt.legend(['loss', 'val_loss'])
plt.show()

plt.figure(figsize=(12,8))
plt.plot(hist.history['acc'])
plt.plot(hist.history['val_acc'])
plt.legend(['acc', 'val_acc'])
plt.grid()
plt.show()

loss, accuracy = model.evaluate(X_test, y_test)
print("Accuracy = {:.2f}".format(accuracy))
```

## 1.4 Titanic 생존자 데이터 시각화



- 타이타닉 호에 탑승한 승객들에 관한 정보를 담고 있는 데이터.
- 기계 학습과 데이터 분석 연습용으로 널리 사용되며, 주로 생존 여부를 예측하는 문제에 활용.

## 1.4 Titanic 생존자 데이터 시각화

- **survived** (생존 여부):

0: 사망

1: 생존

- **pclass** (객실 등급):

1: 1등급

2: 2등급

3: 3등급

- **sex** (성별):

male: 남성

female: 여성

- **age** (나이):

승객의 나이

- **sibsp** (형제자매 및 배우자 수):

함께 탑승한 형제자매 및 배우자의 수

- **parch** (부모 및 자녀 수):

함께 탑승한 부모 및 자녀의 수

- **fare** (운임 요금):

지불한 운임 요금

- **embarked** (탑승 항구):

C: Cherbourg

Q: Queenstown

S: Southampton

- **class** (객실 등급, pclass와 동일):

First: 1등급

Second: 2등급

Third: 3등급

- **who** (승객 유형):

man: 남성

woman: 여성

child: 어린이

- **adult\_male** (성인 남성 여부):

True: 성인 남성

False: 성인 남성이 아님

- **deck** (객실 번호):

객실 번호가 알려져 있지 않은 경우 NaN으로 표시

- **embark\_town** (탑승 항구의 도시):

Cherbourg, Queenstown, Southampton

- **alive** (생존 여부, survived와 동일):

yes: 생존

no: 사망

- **alone** (혼자 여부):

True: 혼자 탑승

False: 가족과 함께 탑승

## 1.4 Titanic 생존자 데이터 시각화

### 1. Module 및 data load

```
import seaborn as sns
import matplotlib.pyplot as plt

# Seaborn의 내장 데이터셋인 Titanic 데이터 로드
titanic = sns.load_dataset('titanic')
titanic
```

## 1.4 Titanic 생존자 데이터 시각화

### 2. 다양한 시각화를 통한 데이터 탐색

# Countplot: 성별에 따른 생존자 및 사망자 수 시각화

```
plt.figure(figsize=(8, 6))
sns.countplot(x='survived', hue='sex', data=titanic)
plt.title('Survival Count by Gender')
plt.xlabel('Survived')
plt.ylabel('Count')
plt.show()
```

# Barplot: 객실 등급에 따른 평균 나이 시각화

```
plt.figure(figsize=(8, 6))
sns.barplot(x='class', y='age', data=titanic, ci=None)
plt.title('Average Age by Passenger Class')
plt.xlabel('Passenger Class')
plt.ylabel('Average Age')
plt.show()
```



## 1.4 Titanic 생존자 데이터 시각화

### 2. 다양한 시각화를 통한 데이터 탐색

# FacetGrid: 객실 등급 및 성별에 따른 생존 여부 시각화

```
g = sns.FacetGrid(titanic, col='survived', row='sex', margin_titles=True, height=3)
g.map(plt.hist, 'age', bins=20, color='purple')
g.set_axis_labels('Age', 'Count')
g.set_titles(col_template='{col_name} columns', row_template='{row_name} rows')
plt.show()
```

# Pointplot: 객실 등급, 성별에 따른 생존률 시각화

```
plt.figure(figsize=(10, 6))
sns.pointplot(x='class', y='survived', hue='sex', data=titanic, markers=['o', '^'],
linestyles=['-', '--'])
plt.title('Survival Rate by Passenger Class and Gender')
plt.xlabel('Passenger Class')
plt.ylabel('Survival Rate')
plt.show()
```

**감사합니다**